

CSC1003 Tutorial 5

Recursion

Yuanqing Ma

yuanqingma@link.cuhk.edu.cn

November 3, 2025

1. Brief Introduction of Recursion

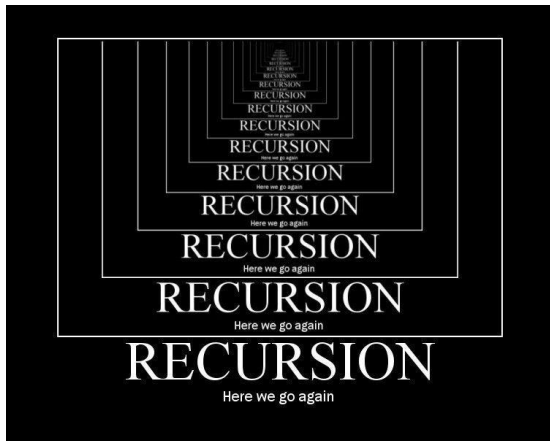
- 1.1 What is recursion?
- 1.2 When to use recursion?
- 1.3 How to use recursion?

2. Some Examples

Brief Introduction of Recursion

What is recursion?

- Break a problem into smaller and smaller subproblems
- Eventually reaching the base case
- **A function calling itself**



When to use recursion?

For a relatively large problem,

- Already have done something on a part of it,
- The remaining could be solved by similar procedures used before

⇒ **Use Recursion!**

How to use recursion?

1. A recursive algorithm must have a **base case**.
2. A recursive algorithm must change its state and **move towards the base case**.
3. A recursive algorithm must call itself **recursively**.

Some Examples

Example: Fibonacci Sequences

- In Fibonacci sequences, the next number is the sum of the previous two numbers
- Example: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55...
- $F(0) = 0; F(1) = 1; F(n) = F(n-1) + F(n-2), n \geq 2$

Recursion:

- Base Case: $F(0) = 0$ and $F(1) = 1$
- Recursive case: For $n \geq 2$, $F(n) = F(n-1) + F(n-2)$

Example: Fibonacci Function

```
public static int fibonacci(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    int n0 = 0;  
    int n1 = 1;  
    for (int i = 2; i <= n; i++) {  
        n1 = n0 + n1;  
        n0 = n1 - n0;  
    }  
    return n1;  
}
```

Faster, but too simple to solve a more complex problem

Example: Fibonacci Function

```
public static int fibonacci(int n) {  
    if (n == 0) {  
        return 0;  
    } else if (n == 1) {  
        return 1;  
    }  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

Relatively slower and more space-consuming, but able to solve more complex problems.
(Faster code in lecture slides.)

Example: Factorial

The factorial of a number n is the product of all positive integers less than or equal to n :

$$n! = n \times (n - 1) \times (n - 2) \times \cdots \times 1$$

- Base case: $F(0) = 1$
- Recursive case: For $n \geq 1$, $F(n) = n \times F(n - 1)$

Example: Factorial

```
public static int factorial(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return n * factorial(n - 1)  
}
```

Example: Greatest Common Divisor (GCD)

Definition

c is a **common divisor** of a and b means a is a multiple of c and b is a multiple of c .
The **greatest common divisor** of a and b , namely $\gcd(a, b)$, is defined as the greatest number among all common divisors of a and b .

Examples: $\gcd(8, 10) = 2$, $\gcd(10, 30) = 10$, $\gcd(3, 11) = 1$

Example: GCD

How to compute the gcd of 2 positive integers?

Divisor

- Write a Java program to compute the greatest common divisor of two given positive integers x and y . For example, the highest common factor of 54 and 24 is 6.
- OJ link: https://oj.cuhk.edu.cn/d/csc1003_2025_fall/p/P1008

Euclid's Algorithm

Theorem

Let $x \geq y > 0$, then $\gcd(x, y) = \gcd(y, x \% y)$

Example: $\gcd(16, 10)$

Calculate GCD

Assumes the inputs are natural numbers x, y satisfying $x \geq y \geq 0$ and $x > 0$

```
public static int gcd(int x, int y) {  
    if (y == 0) {  
        return x;  
    }  
    return gcd(y, x % y);  
}
```

Practice: Power

How can we compute the power of a number x^n ?

```
public static double power(double x, int n) {  
    if (n == 0) {  
        return 1;  
    }  
    if (n < 0) {  
        return 1.0 / power(x, -n);  
    }  
    // TODO: Implement the recursive case  
}
```

Practice: Power

An approach may be keep multiplying x.

```
public static double power(double x, int n) {  
    if (n == 0) {  
        return 1;  
    }  
    if (n < 0) {  
        return 1.0 / power(x, -n);  
    }  
    return x * power(x, n - 1);  
}
```

Practice: Power

Can we do better than this?

(**Hint:** Consider the case that n is even. Then we can actually compute $\text{power}(x, n / 2)$ and square it, which is more efficient than keep multiplying x for $n / 2$ times. How about the case that n is odd?)

Practice: Power

An optimized approach:

```
public static double power(double x, int n) {  
    ...  
  
    double halfPower = power(x, n / 2);  
    if (n % 2 == 0) {  
        // Even exponent:  $x^n = (x^{(n/2)})^2$   
        return halfPower * halfPower;  
    } else {  
        // Odd exponent:  $x^n = x * (x^{((n-1)/2)})^2$   
        return x * halfPower * halfPower;  
    }  
}
```

Actually, after learning lecture 7, you can find out that this method achieves a faster complexity ($O(\log n)$) than the previous approach ($O(n)$).

Practice: Digit Distance

For an integer, the **digit distance** is the sum of the absolute differences between consecutive digits. For example:

- The digit distance of 61 is 5, as the absolute value of $6 - 1$ is 5.
- The digit distance of 71253 is 12 ($|7 - 1| + |1 - 2| + |2 - 5| + |5 - 3| = 6 + 1 + 3 + 2$).
- The digit distance of 6 is 0 because there are no pairs of consecutive digits.

Write a function that determines the digit distance of a positive integer. Try to use recursion to do the job. (**Hint:** You might need helper functions!)

Practice: Digit Distance

- Starter code available on blackboard. You can find it under Tutorial/Tutorial_5_Recursion/tut5_starter_code.zip
- **Remember to place the two files (Sample.java, DigitDistance.java) in the same directory** before running DigitDistance.java to check whether your implementation is correct.
- You can check out a sample solution in the Sample.java. Note that there are multiple ways to implement the function using recursion, and the sample is just for your reference.

Q&A